GOSIM

GOSIM Shenzhen 2026

October 16-17, 2026 Shenzhen, China



GOSIM

GOSIM Shenzhen 2026

全球边缘的智能体 AI

基于 SPEAR 与 MCP 部署运维智能体

Wilson Wang, InfiniEdge AI TSC Member,
ByteDance



CONTENTS

Part 01 AI和智能体在边缘的现状

Part 02 智能体在边缘的编排形态

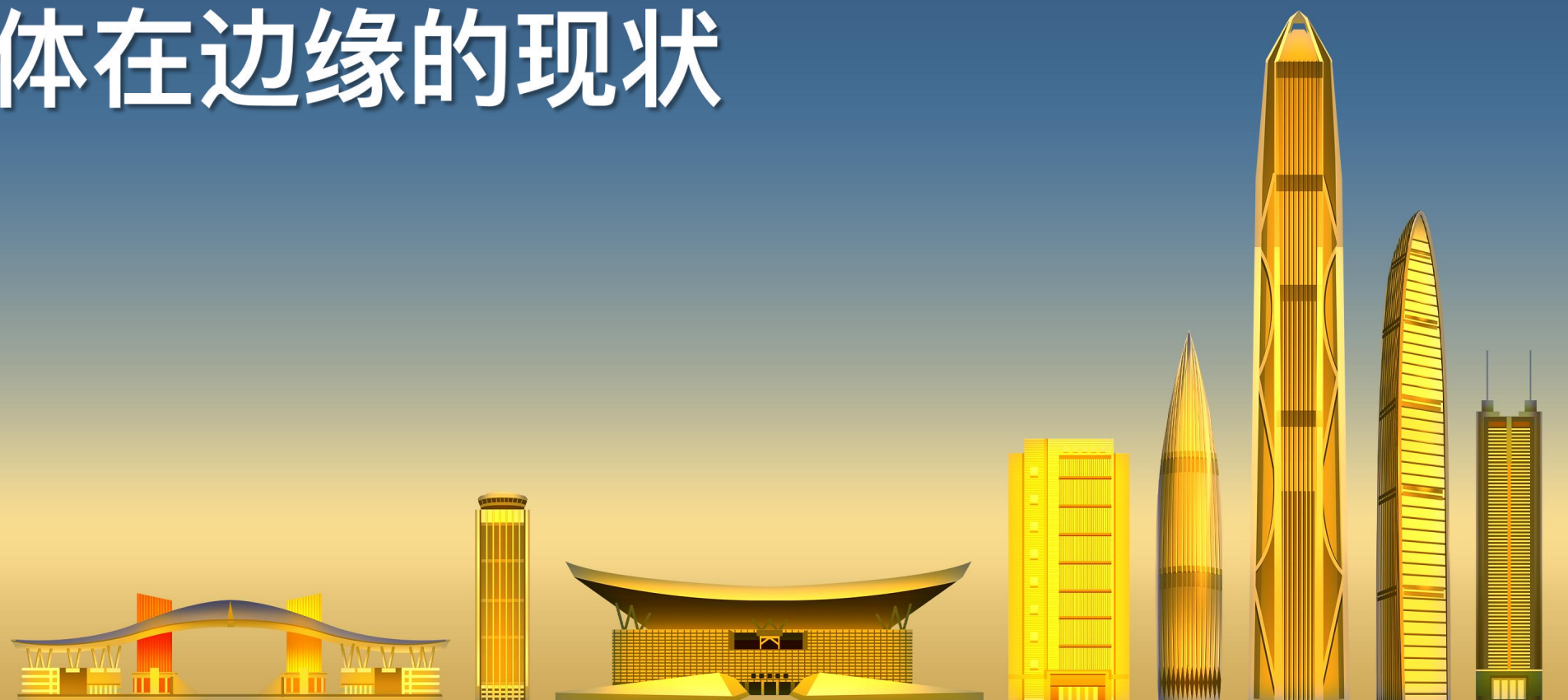
Part 03 智能体的执行和交互

Part 04 SPEAR框架介绍和未来计划



01

AI和智能体在边缘的现状



边缘业务侧：AI使用有限

1. AI还未在远边缘(Far Edge)全面铺开，AI推理还是以云上为主。
   原因：算力缺口导致AI能力还没有下沉到边缘
2. 在近边缘（Near Edge），AI应用场景较为有限。
   使用的也是较为简单的模型，LLM还没有到可以产品化的程度。

边缘运维侧：开始全面拥抱AI

AI使得所需运维人力大幅下降，内部旧运维系统开始被新的基于AI的运维系统替代。一个SRE团队（10人左右）三年开发的系统，基本功能可以被现在一个用AI的工程师一两个月左右重写。
这类系统的特点：
1. 对于速度和性能要求不高，更多是工作流类型
2. 对于灵活性要求高，能够快速POC，能自我进化和闭环

02

智能体在边缘运维的编排形态



边缘环境的常态

1. 资源不够是常态：过多内存预留导致边缘本不富裕的资源雪上加霜
2. 线路出问题是常态：自然影响（台风过境、地震突发）和人类影响（船只活动、战争冲突）
3. 机房出问题是常态：电力不足、硬件失效、路由配置错误

现有多数边缘业务主要是VM或者容器，如何在边缘运维部署智能体的同时，减轻对于现有业务的影响？



为了在边缘编排为运维使用的智能体，我们需要：

1. 轻量运行时：减少资源开销，留给业务更多资源使用。（VM和容器都太重）
2. 动态调度：根据运维需求，选择适合的节点去运行智能体。
3. 灾难恢复：在出现节点失效等情况下，自动重新调度运行相关智能体任务。

结论：我们需要的是一个能够支持分布式智能体编排的Serverless框架



03

智能体的运行时



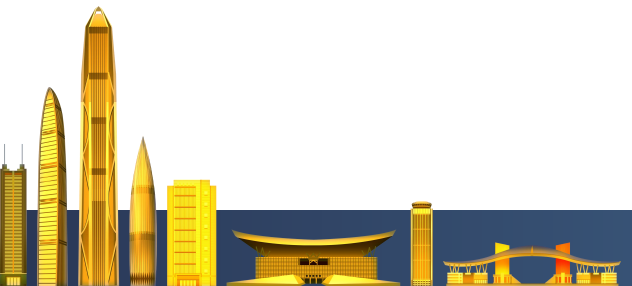
智能体的触发和执行

与Serverless非常类似

- Time (Cron, Timer, Deadline, …)
- Event (MQ, Webhook, Git, …)
- State (Threshold, Data Change, …)
- Manual

和Serverless不同

- 智能体通常是Stateful的，使用各种不同的记忆和存储
- 执行时长分布广，可长可短



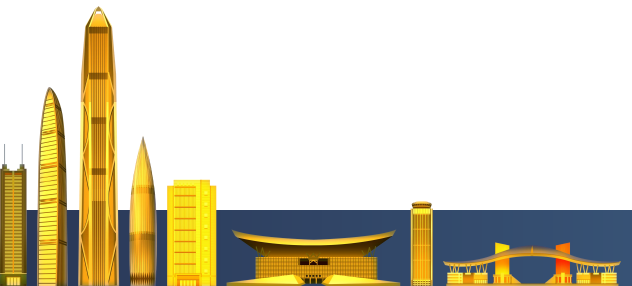
智能体间的特有交互

区别于传统的Serverless之间的交互，智能体交互目前有：

- A2A
- MCP

但是，有一系列的局限：

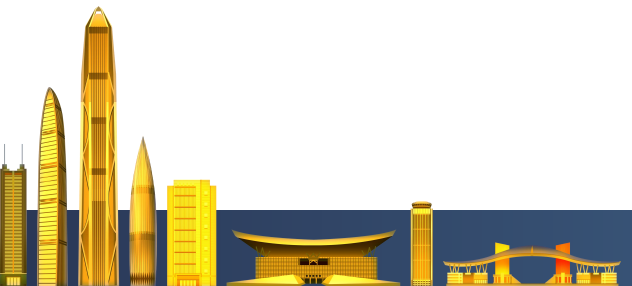
- 对于Agent的开发来过重
- 过多和Agent开发本身无关的底层技术细节
- 版本不稳定



如何选择一个轻量高效的智能体框架来满足我们运维边缘集群的需求？

我们选择的解决方案：

SPEAR https://github.com/lfedgeai/SPEAR



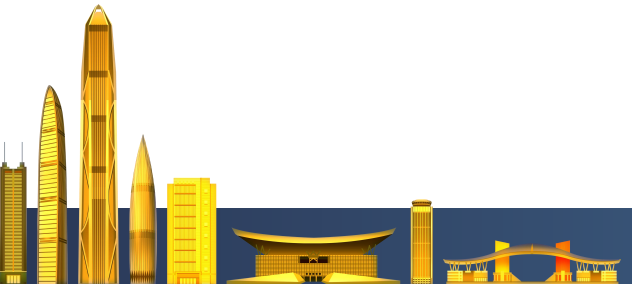
04

SPEAR框架介绍和路线图

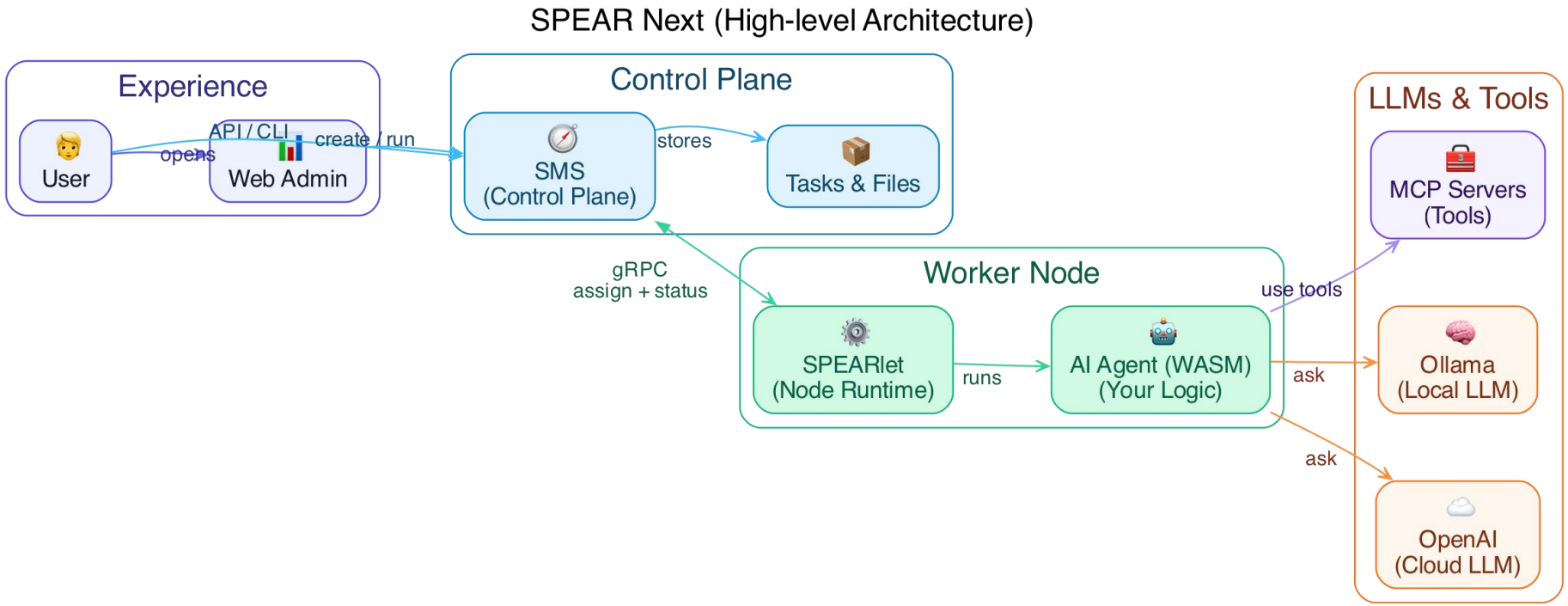


什么是SPEAR

- LF Edge 组织下的 InfiniEdge AI 项目一个工作流
- 目标是实现一个分布式智能体运行框架
  - 提供操作系统级原语（通过WebAssembly Hostcall实现）
  - 实现函数级别分布式执行（类Ray）



- Master-Worker集群形式（类似K8s）
- Worker上运行基于WebAssembly的Agent



为什么用SPEAR

- 多语言支持（目前提供js、rust、c的sdk）
- 使用简单，目前支持
  - Chat completion及function call(包括MCP)
  - ASR
- 底层实现了一套Async Hostcall，可以理解为一个为Agent量身定做的POSIX

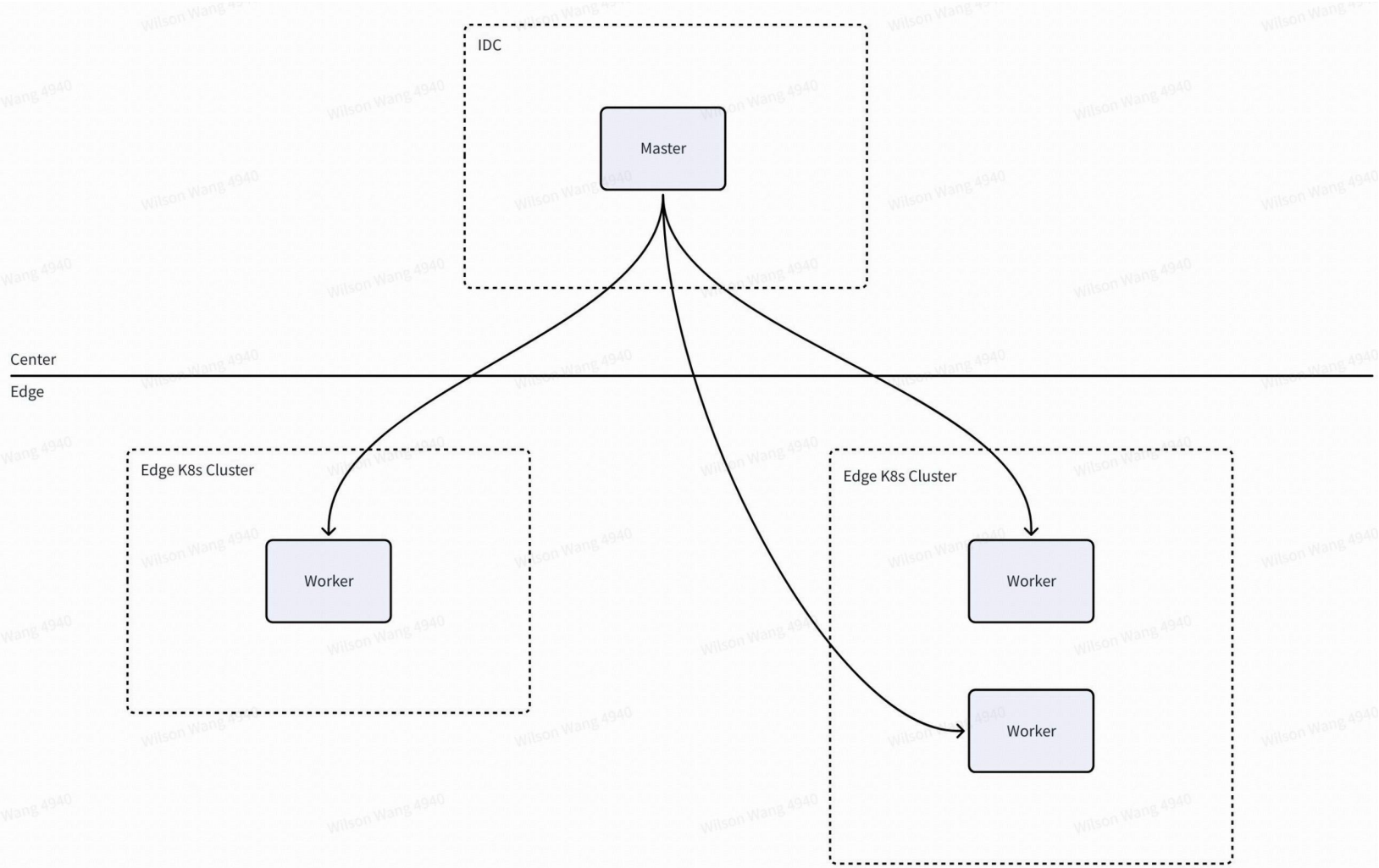
```
import { Spear } from "spear";

export default async function main() {
  const tools = [
    Spear.tool({
      name: "sum",
      description: "Add two integers",
      parameters: {
        type: "object",
        properties: {
          a: { type: "integer" },
          b: { type: "integer" },
        },
        required: ["a", "b"],
      },
      handler: ({ a, b }) => {
        console.log("sum invoked:", "a=", a, "b=", b);
        return { sum: (a ?? 0) + (b ?? 0) };
      },
    }),
  ];

  const resp = await Spear.chat.completions.create({
    model: "gpt-4o-mini",
    messages: [{ role: "user", content: "Please call sum(a,b) for a=7 and b=35." }],
    tools,
    maxTotalToolCalls: 4,
    maxIterations: 4,
  });

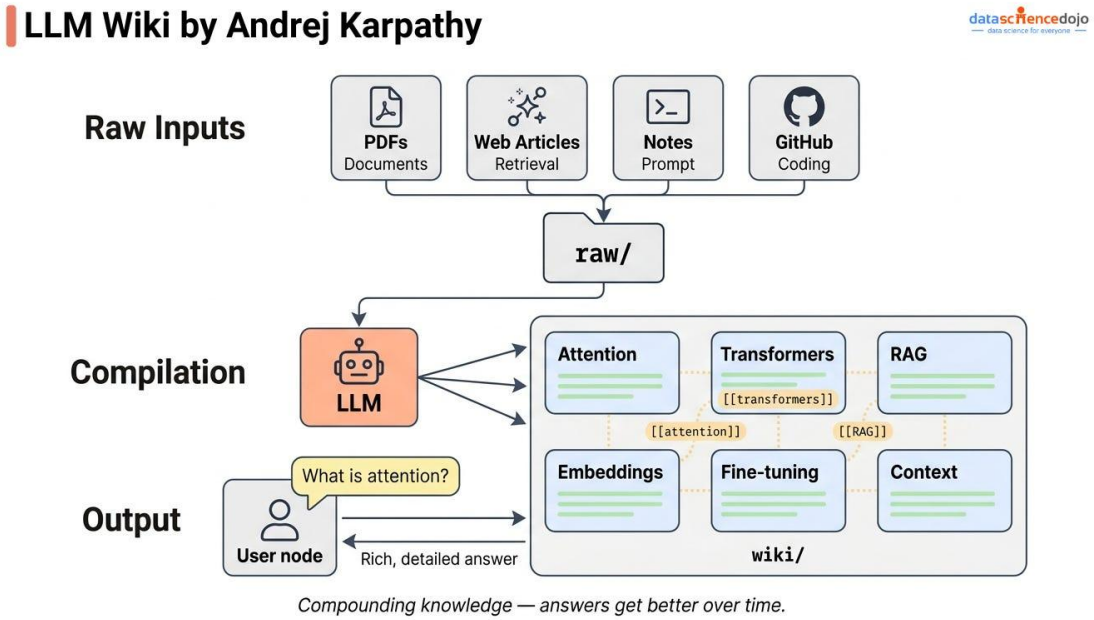
  return resp.text();
}
```

SPEAR边缘部署示例



LLM的能力在最近半年到一年有了比较大的提升，LLM能够完成很多之前不能完成的工作。但是对于复杂的任务，LLM仍然有可能因为幻觉而错误选择解决方案。**如何能够减少LLM幻觉?**在实践中，我们发现通过知识“编译”生成的**知识库是一个对于LLM处理运维问题的一个非常重要的补充**。大量的团队已经开始在运维环境中使用知识库配合LLM并且收到了非常好的效果。我们计划**在SPEAR中支持知识库的能力**，便于开发者使用。

1. LLMWiki from Andrej Karpathy
   https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f
2. GBrain from Garry Tan
   https://github.com/garrytan/gbrain



GOSIM

GOSIM Shenzhen 2026

Thanks!



GOSIM

GOSIM Shenzhen 2026

Thanks!



GOSIM

GOSIM Shenzhen 2026

Thanks!

